



Multi-Cloud Portfolio — System Architecture Blueprint

Author: Senior Architect Review
Date: 1 October 2026
Status: Architecture Decision Baseline — Owner Decisions Applied
Scope: 5 projects × 3 clouds × 26 AI concepts

[!IMPORTANT] This document captures **architecture decisions, component designs, data flows, and acceptance criteria** for all six projects. It supersedes the original strategy map and complements the existing Rahul Site LLD. Each section is structured for a development team to pick up and execute.

Table of Contents

- 1. [Architecture Philosophy](#)
- 2. [Global Design Decisions](#)
- 3. [Project 1 — Rahul Site \(Azure\)](#)
- 4. [Project 2 — Creator Vault \(AWS\)](#)
- 5. [Project 3 — Stock Site \(GCP\)](#)
- 6. [Project 4 — Kids App \(GCP Firebase\)](#)
- 7. [Project 5 — Folder Manager \(Azure\)](#)
- 8. [Cross-Cutting Concerns](#)
- 9. [Infrastructure Strategy](#)
- 10. [MCP Integration Architecture](#)
- 11. [Master Build Sequence](#)

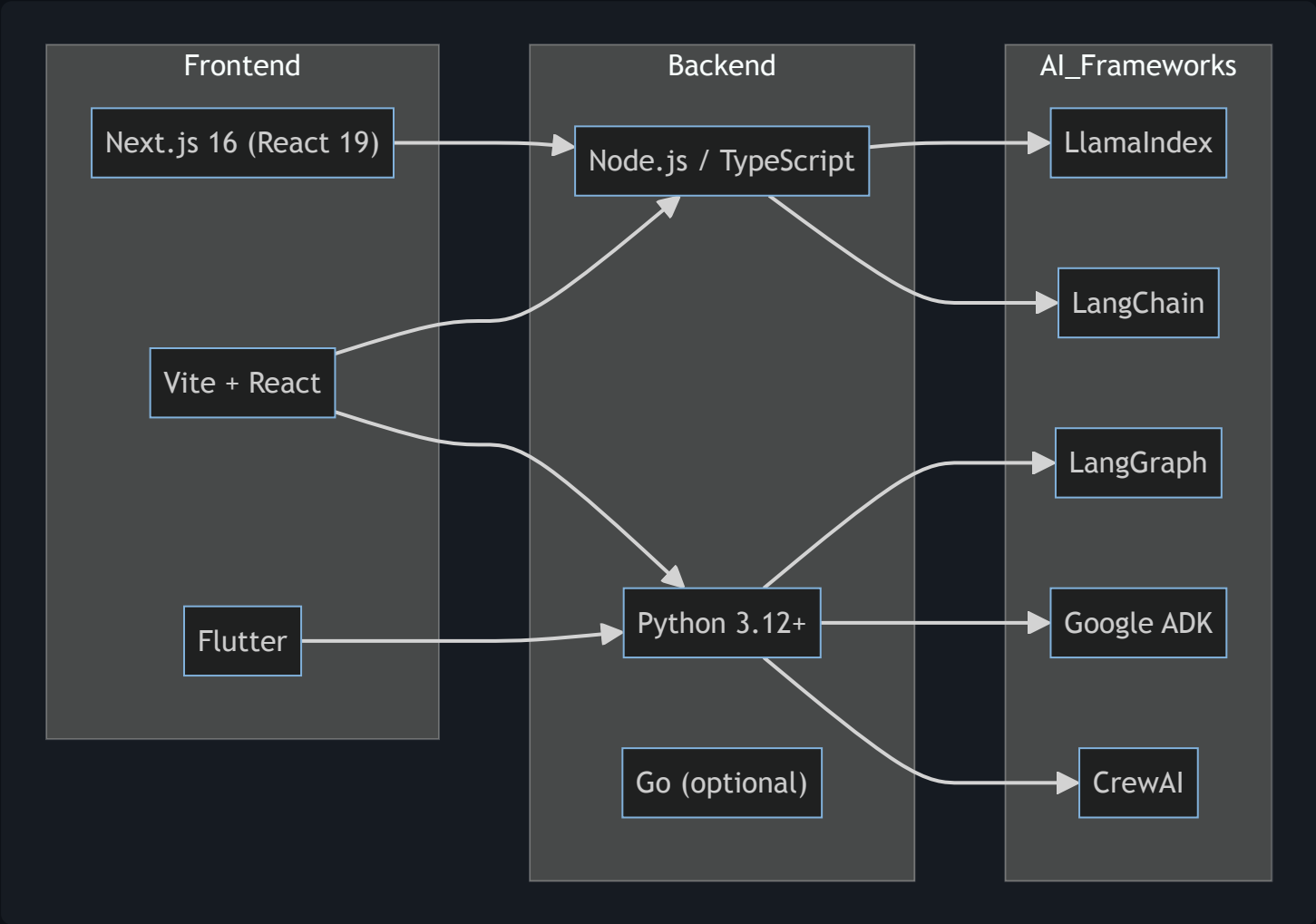
1. Architecture Philosophy

Core Principles

Principle	Rationale
Cloud-Native First	Each project uses the native services of its target cloud — no abstraction layers that hide cloud-specific learning
Monorepo Infra, Polyrepo Apps	One <code>infrastructure-deployments</code> repo for all Terraform; separate Git repos per application for independent CI/CD

Principle	Rationale
AI as a First-Class Citizen	Every project teaches specific AI concepts; the AI pipeline is not bolted on but designed into the architecture
Progressive Complexity	Build order moves from familiar (Azure, existing code) → new territory (AWS serverless) → advanced (GCP multi-agent)
Production-Grade from Day 1	Security, observability, error handling, and testing are not optional phases — they ship with v1
MCP as the Universal Glue	Every project exposes an MCP server, enabling cross-project AI agent interop

Technology Alignment Matrix



2. Global Design Decisions

ADR Index (Global)

ID	Decision	Resolution	Consequence
G-ADR-01	IaC tool	Terraform with cloud-specific providers	Consistent workflow; modules per cloud; state in remote backend per cloud
G-ADR-02	CI/CD	GitHub Actions with OIDC federation per cloud	No static credentials; separate workflows per cloud provider
G-ADR-03	MCP protocol version	MCP v1.0 (2025 spec)	Standardized tool/resource/prompt primitives across all servers
G-ADR-04	Secrets management	Cloud-native vaults (Key Vault / Secrets Manager / Secret Manager)	No <code>.env</code> files in production; local dev uses <code>.env.local</code> with dummy values
G-ADR-05	Observability	Cloud-native (App Insights / CloudWatch / Cloud Logging) + structured JSON logs	Unified log schema: <code>{requestId, service, operation, duration, status, error?}</code>
G-ADR-06	API design	REST for CRUD; WebSocket/SSE for streaming; gRPC reserved for inter-agent comms	Pragmatic choice; AI streaming needs persistent connections
G-ADR-07	Testing baseline	Unit (Vitest/pytest) + Integration + E2E (Playwright) per project	No project ships without passing test gates
G-ADR-08	AI model strategy	Use cloud-native models first (Azure OpenAI, Bedrock, Vertex AI)	Maximizes learning; avoids third-party API key sprawl
G-ADR-09	Monorepo infra state	Remote backends: Azure Storage (Azure projects), S3 (AWS projects), GCS (GCP projects)	State isolation per cloud; no cross-cloud state dependencies
G-ADR-10	Environment strategy	<code>dev</code> → <code>staging</code> → <code>prod</code> per project; feature branches deploy to <code>dev</code>	Cost-managed: <code>dev</code> is ephemeral; <code>staging</code> mirrors prod config

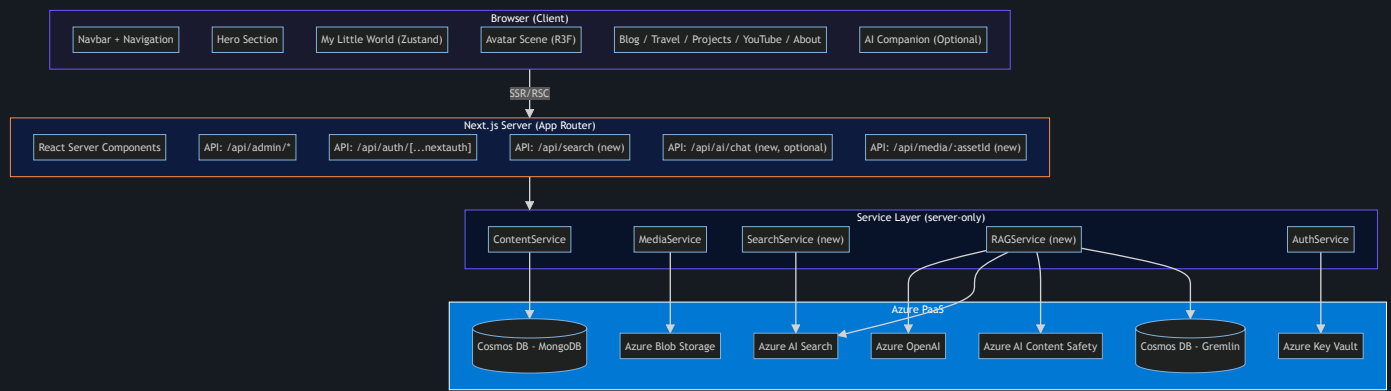
Shared Patterns

3. Project 1 — Rahul Site (Azure)

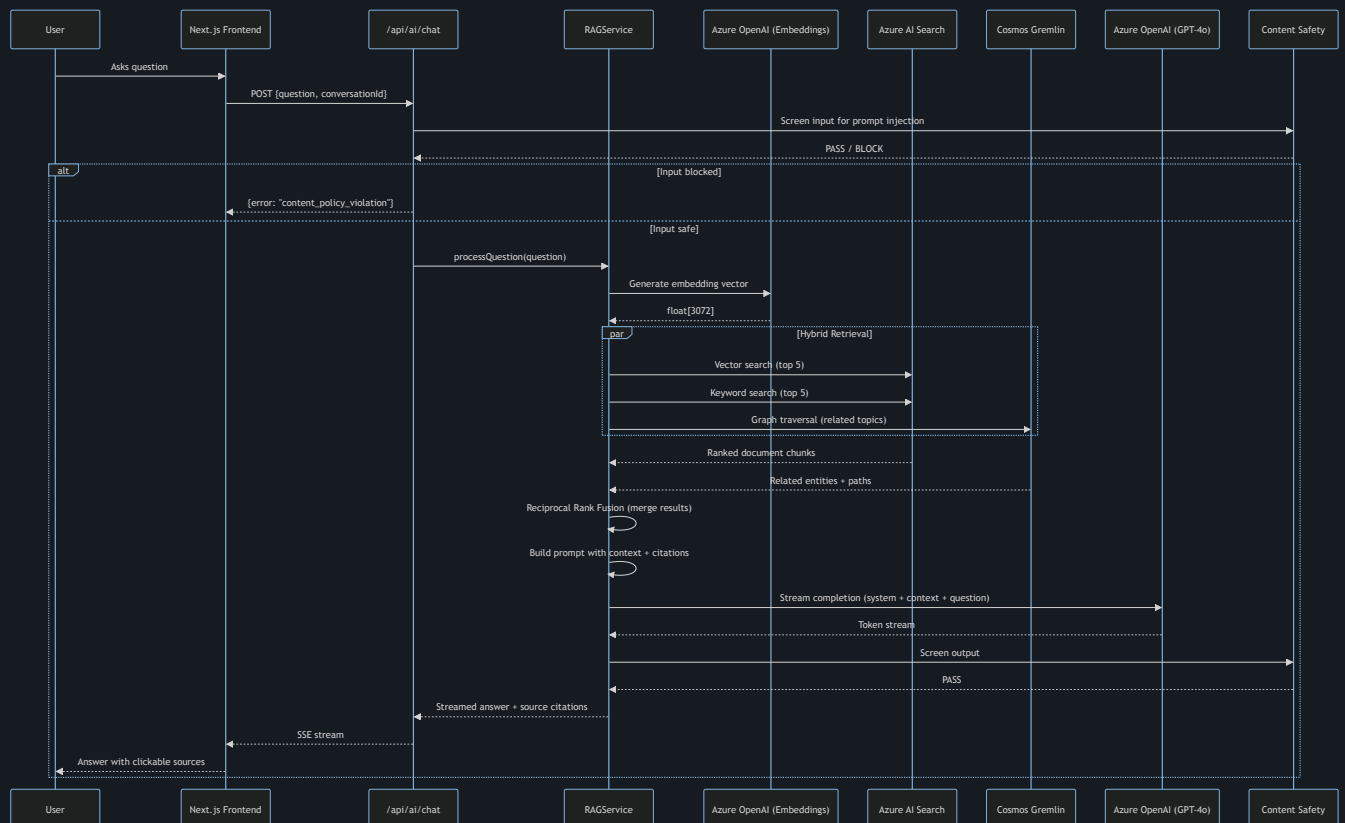
3.1 Design Decision Summary

Decision	Choice	Why
Framework	Next.js 16.3.2 / React 19 (existing)	Preserve investment; SSR + ISR for SEO; App Router for streaming
Database	Cosmos DB for MongoDB (existing)	Already provisioned; Mongoose ORM; free-tier friendly
Auth	NextAuth v4 + Entra ID / Google (existing)	SSO for admin; JWT sessions; no custom password auth
AI Search	Azure AI Search (new)	Native hybrid search (vector + keyword); integrates with Azure OpenAI
AI Model	Azure OpenAI GPT-4o + text-embedding-3-large (new)	RAG pipeline; prompt engineering; same-cloud latency
Knowledge Graph	Cosmos DB Gremlin API (new, separate account)	Graph queries for cross-document reasoning; stays in Azure
Content Safety	Azure AI Content Safety (new)	Guardrails for public-facing AI responses
RAG Framework	LlamaIndex (TypeScript)	Best-in-class for RAG; native Azure integrations
CSS	Tailwind CSS v4 (existing)	Already in project; custom design tokens defined
State	Zustand v5 (existing)	Client-side world preferences; lightweight
3D	React Three Fiber + Drei (existing)	Avatar scene; lazy-loaded; graceful fallback
Hosting	Azure App Service B1 (existing)	GitHub Actions OIDC deploy; shared plan

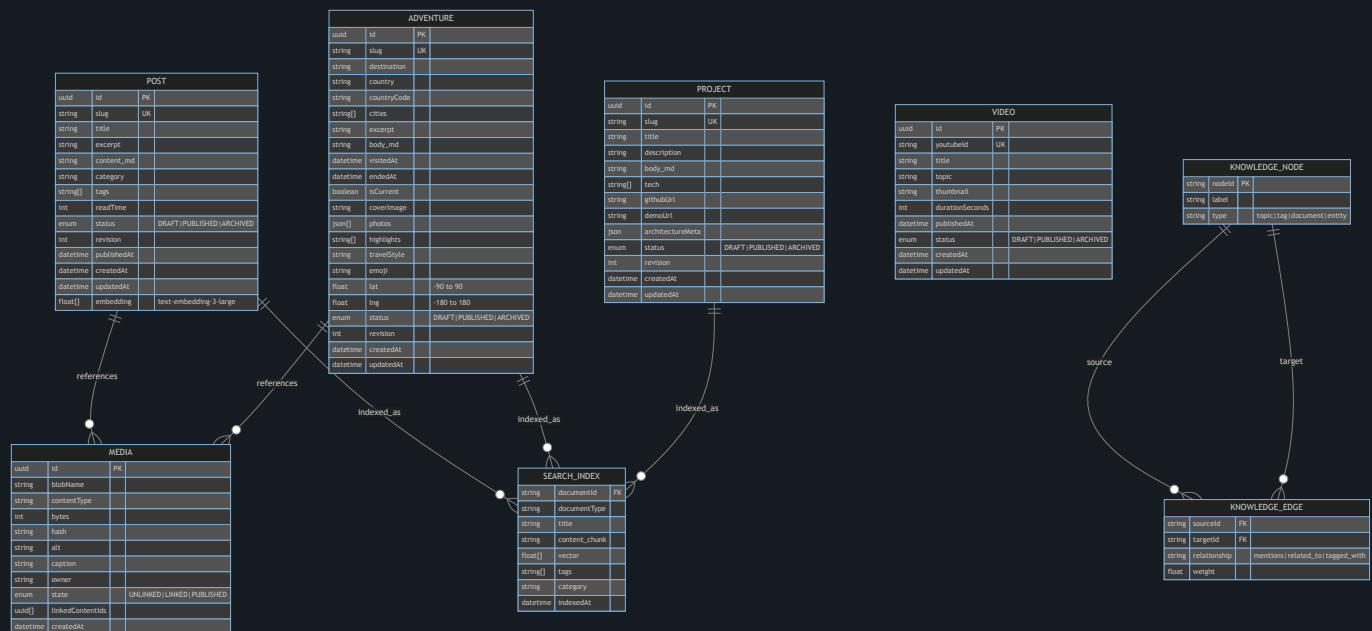
3.2 Component Architecture



3.3 RAG Pipeline Design



3.4 Data Model (Evolution from Existing)



3.5 Implementation Phases

Phase	Scope	Duration	Dependencies
P1: Foundation Fix	Close G-01 through G-10 gaps from LLD; runtime validation (Zod); real 404s; fixture removal; auth hardening	2-3 weeks	None
P2: Content Service	Server-only published content repository; DB reads for public pages; cache invalidation; RSS/sitemap from real data	1-2 weeks	P1
P3: Media Pipeline	Secure upload with byte-signature validation; blob proxy for private container; Media model; linked asset tracking	1-2 weeks	P1
P4: Search & Embeddings	Azure AI Search index; embedding generation on publish; hybrid search API; public search UI	2 weeks	P2
P5: RAG Pipeline	LlamaIndex integration; vector + keyword retrieval; prompt engineering; streaming response; citation formatting	2-3 weeks	P4
P6: Knowledge Graph	Cosmos Gremlin provisioning; auto-extraction of topics/entities on publish; graph-augmented retrieval	2 weeks	P5
P7: Guardrails & Eval	Content Safety integration; prompt injection defense; Ragas evaluation framework; quality metrics dashboard	1-2 weeks	P5

Phase	Scope	Duration	Dependencies
P8: MCP Server	<code>rahul-knowledge-mcp</code> exposing search + Q&A tools; MCP test client	1 week	P5
P9: Polish & Release	Performance budgets; accessibility audit; visual refinement; E2E tests; deployment verification	2 weeks	All

3.6 Acceptance Criteria

Cross-references existing [need-and-acceptance.md](#) IDs.

ID	Criterion	Verification Method
RS-AI-01	RAG answers cite at least one published source document for factual claims	Automated eval: 50-question test set; faithfulness score ≥ 0.85 (Ragas)
RS-AI-02	AI never invents biography, destinations, dates, or metrics about Rahul	Adversarial prompt test suite (20 attack prompts); zero fabrication tolerance
RS-AI-03	Prompt injection attempts are blocked before reaching LLM	Injection test corpus (OWASP LLM Top 10); Content Safety blocks $\geq 95\%$
RS-AI-04	AI response latency $p95 \leq 5s$ for a 200-token answer	Load test: 10 concurrent users, 100 queries; measured with App Insights
RS-AI-05	Embedding generation completes within 30s of publish action	Integration test: publish $\rightarrow$ verify AI Search index updated
RS-AI-06	MCP server responds to <code>search</code> and <code>ask</code> tools correctly	MCP test client: 10 tool calls; all return valid JSON per MCP spec
RS-AI-07	Knowledge graph has ≥ 3 relationship types traversable	Graph query test: topic $\rightarrow$ document, document $\rightarrow$ tag, tag $\rightarrow$ related_topic
RS-SEARCH-01	Hybrid search returns relevant results for natural language queries	Precision@5 ≥ 0.7 on 30-query evaluation set
RS-SEARCH-02	Search works when AI services are unavailable (keyword fallback)	Kill Azure OpenAI endpoint $\rightarrow$ search still returns keyword results
RS-PERF-01	LCP $\leq 2.5s$ mobile p75; CLS ≤ 0.1 ; INP $\leq 200ms$	Lighthouse CI in pipeline; field data when traffic sufficient
RS-SEC-01	No admin API accessible without valid session	Automated: hit all <code>/api/admin/*</code> routes without auth $\rightarrow$ 401

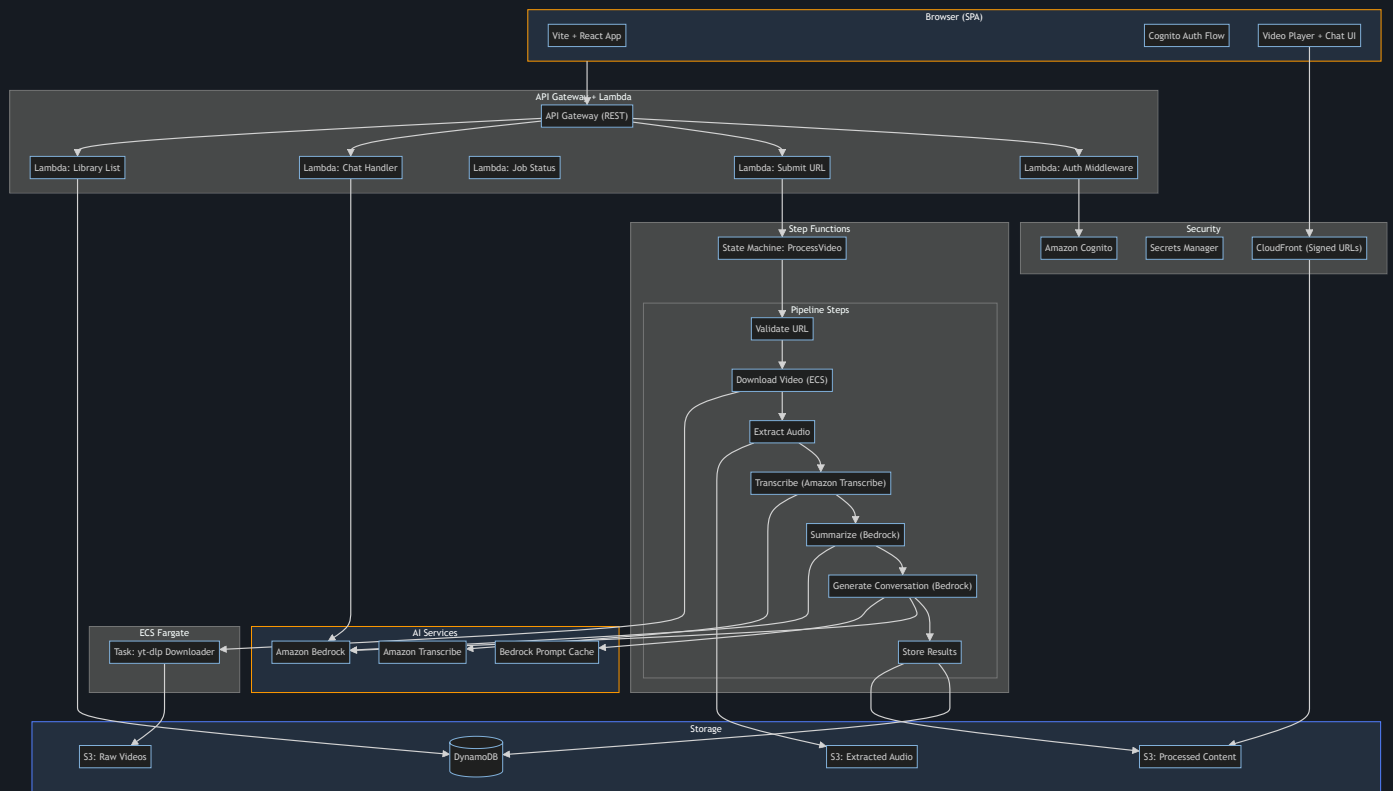
ID	Criterion	Verification Method
RS-SEC-02	AI conversations are not logged with full prompt content	Log audit: verify requestId + truncated query only

4. Project 2 — Creator Vault (AWS)

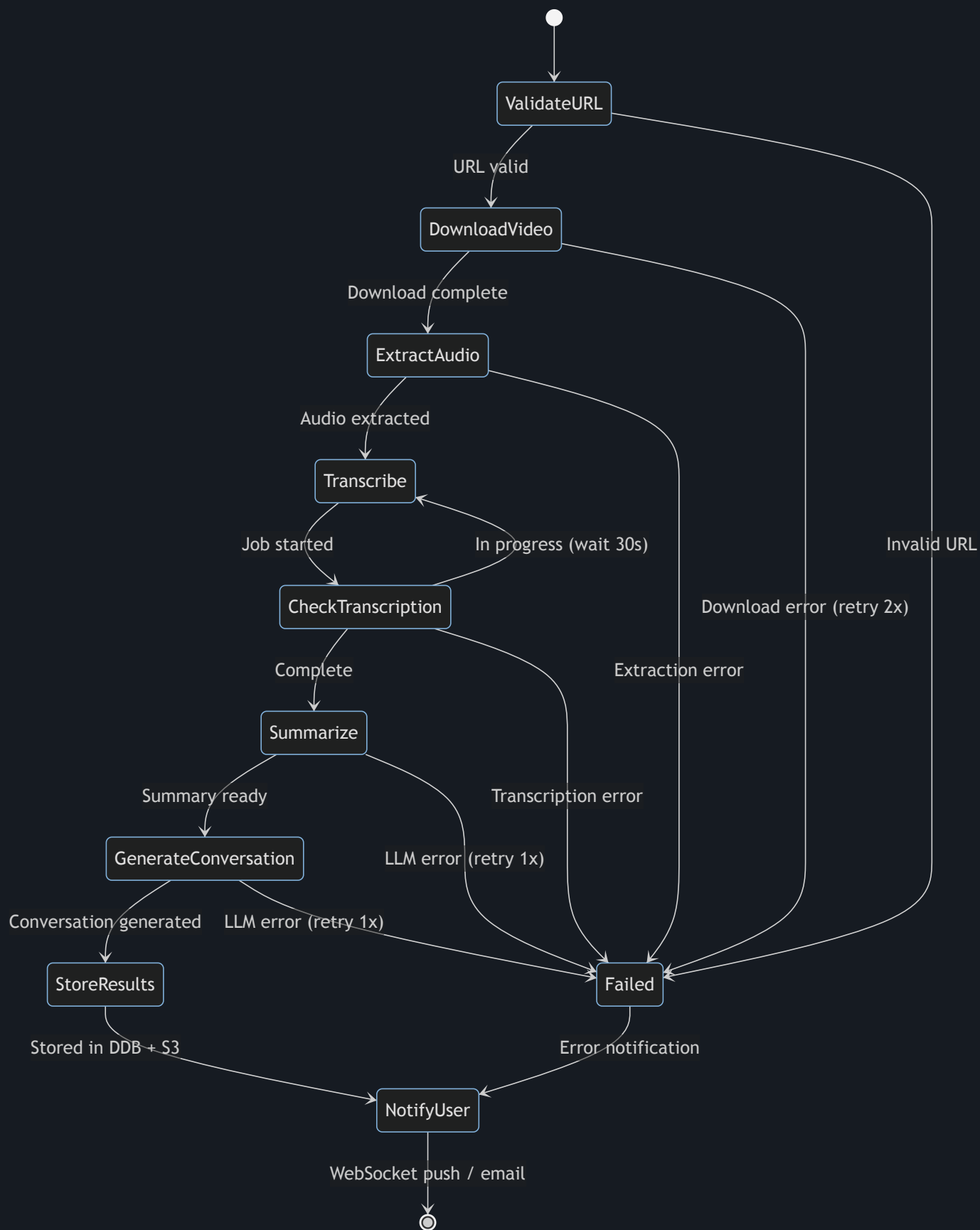
4.1 Design Decision Summary

Decision	Choice	Why
Frontend	Vite + React 19 + TypeScript	Lighter than Next.js; SPA is fine — no SEO needed for private vault
Backend	Python 3.12 (FastAPI)	Best ecosystem for media processing; native AWS SDK; LangChain/LangGraph support
Orchestration	AWS Step Functions	Visual workflow; built-in retry/error handling; native Lambda integration
Compute	Lambda (API) + ECS Fargate (heavy processing)	Lambda for API handlers; Fargate for yt-dlp (needs long runtime + disk)
Database	DynamoDB	Serverless; pay-per-use; flexible schema for metadata + conversations
Storage	S3	Video/audio storage; lifecycle policies for cost management
AI - Transcription	Amazon Transcribe	Native AWS; supports 100+ languages; speaker diarization
AI - LLM	Amazon Bedrock (Claude 4 / Llama)	Managed models; no GPU infrastructure; tool_use API for function calling
AI Framework	LangChain + LangGraph	Tool-calling chains; stateful multi-step workflows with checkpoints
Auth	Amazon Cognito	Native AWS auth; OAuth 2.0; user pools for multi-user support
CDN	CloudFront	Secure signed URLs for media delivery

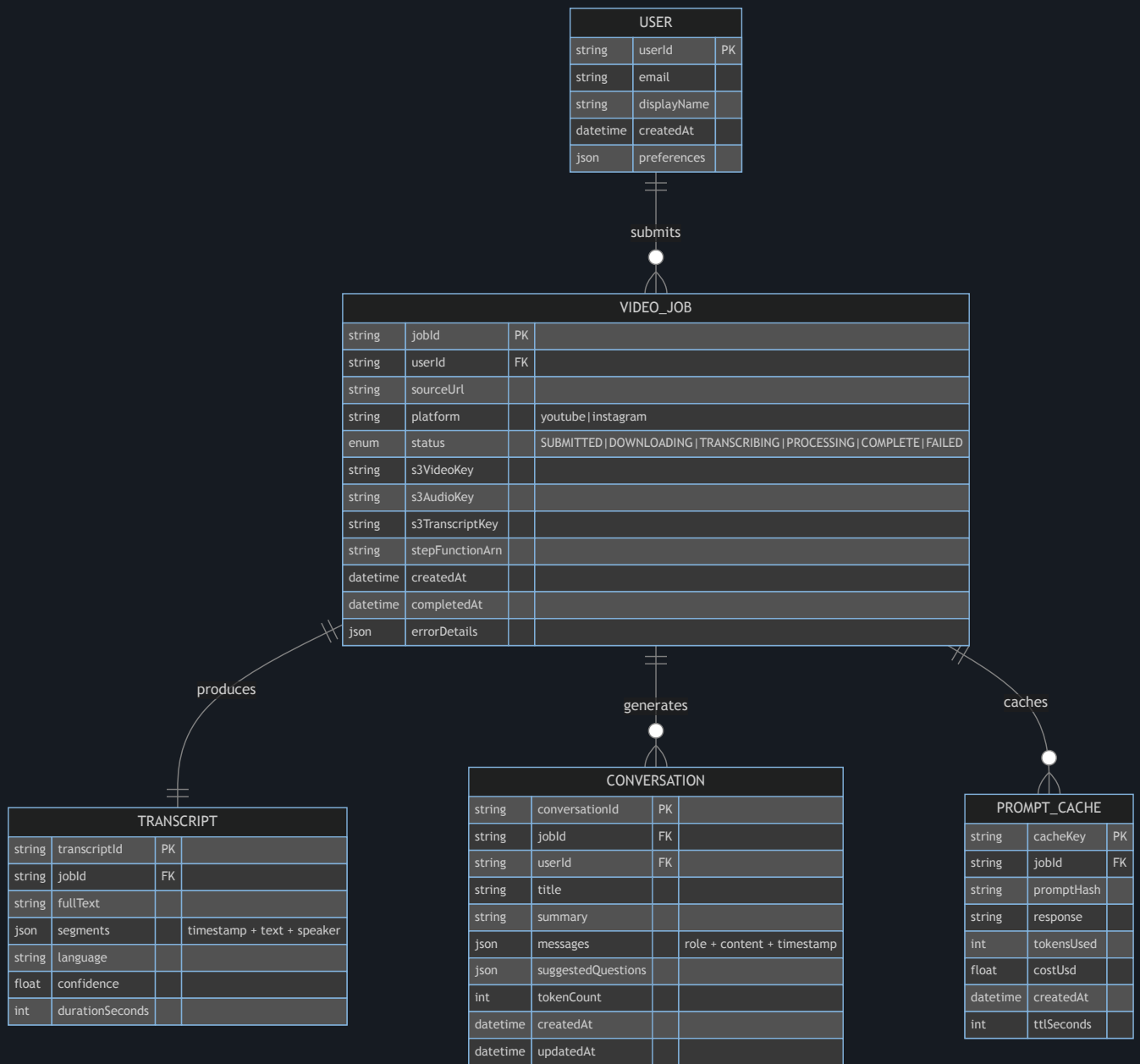
4.2 System Architecture



4.3 Step Functions State Machine



4.4 Data Model



4.5 Implementation Phases

Phase	Scope	Duration
P1: AWS Foundation	Terraform modules (VPC, Lambda, S3, DynamoDB, API Gateway); Cognito user pool; basic CRUD API	2 weeks
P2: Download Pipeline	ECS Fargate task for yt-dlp; S3 lifecycle policies; Step Functions skeleton	2 weeks
P3: Transcription	Amazon Transcribe integration; async job polling; transcript storage; segment parsing	1-2 weeks
P4: LLM Processing	Bedrock Claude integration; summarization prompt; conversation generation; prompt caching	2-3 weeks

Phase	Scope	Duration
P5: Chat Interface	React chat UI; WebSocket streaming; conversation history; suggested questions	2 weeks
P6: Agentic Workflows	LangChain tool-calling chains; LangGraph stateful workflow with checkpoints	2 weeks
P7: MCP Server	<code>creatorvault-mcp</code> exposing transcript search + video summary tools	1 week
P8: Polish	CloudFront delivery; cost monitoring; error handling; E2E tests	2 weeks

4.6 Acceptance Criteria

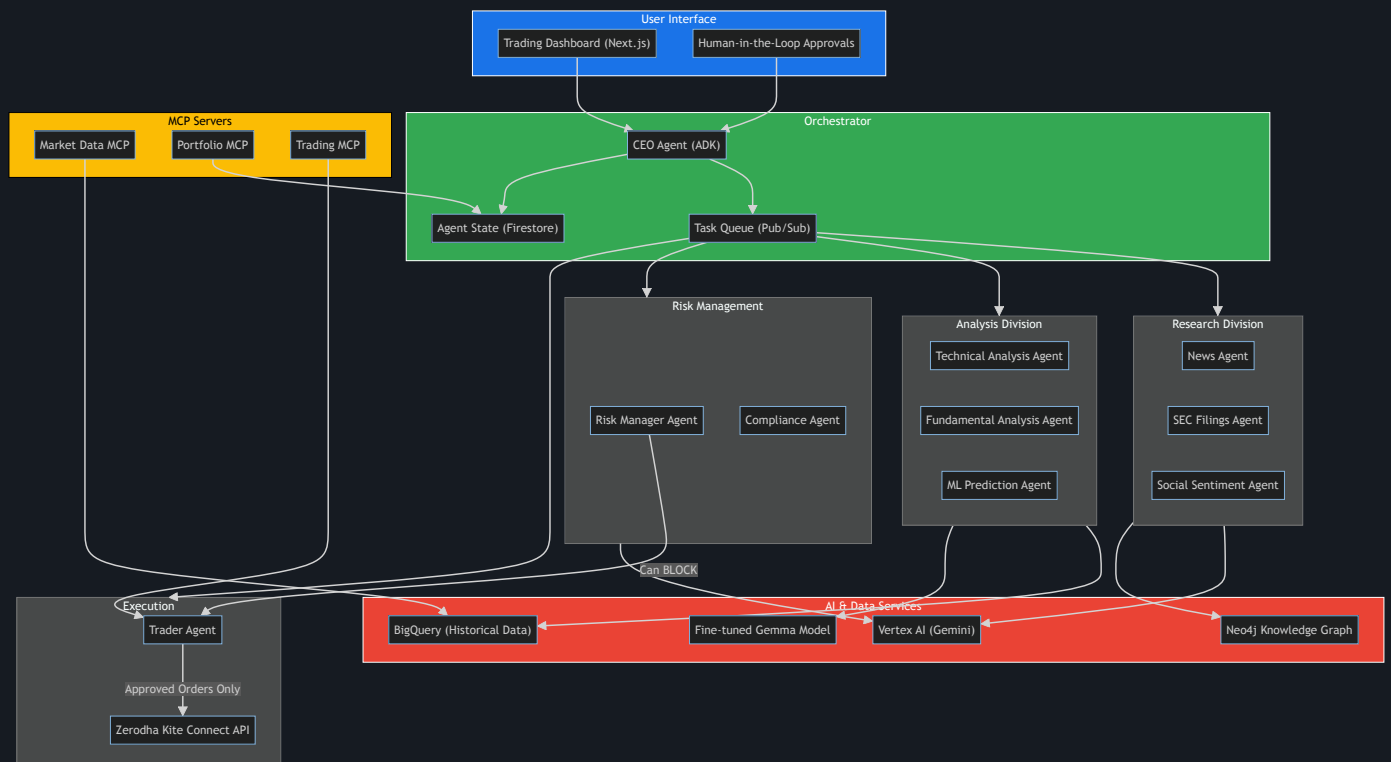
ID	Criterion	Verification
CV-CORE-01	User can paste a YouTube URL and receive a processed transcript within 10 minutes for a 30-min video	E2E test: 5 videos of varying length; all complete within SLA
CV-CORE-02	Transcription accuracy $\geq 90\%$ WER for English content	Compare against manual transcription of 3 test videos
CV-CORE-03	Generated conversation is coherent and covers key topics from the video	Human evaluation: 3 reviewers rate quality $\geq 4/5$
CV-AI-01	Function calling correctly selects tools based on user intent	20-query tool-selection test; accuracy $\geq 95\%$
CV-AI-02	Prompt caching reduces LLM cost by $\geq 40\%$ for repeated queries on same video	Cost comparison: cached vs uncached for 50 queries
CV-AI-03	Streaming response delivers first token within 2s	p95 latency measurement across 50 chat messages
CV-SEC-01	Unauthenticated requests to any API endpoint return 401	Automated: hit all endpoints without Cognito token
CV-SEC-02	Downloaded videos are virus-scanned before processing	S3 event $\rightarrow$ Lambda scan $\rightarrow$ quarantine if detected
CV-SCALE-01	System handles 10 concurrent video processing jobs	Load test with Step Functions concurrency
CV-COST-01	Per-video processing cost $\leq$ \$0.50 for a 30-minute video	Cost tracking dashboard in CloudWatch

5. Project 3 — Stock Site (GCP)

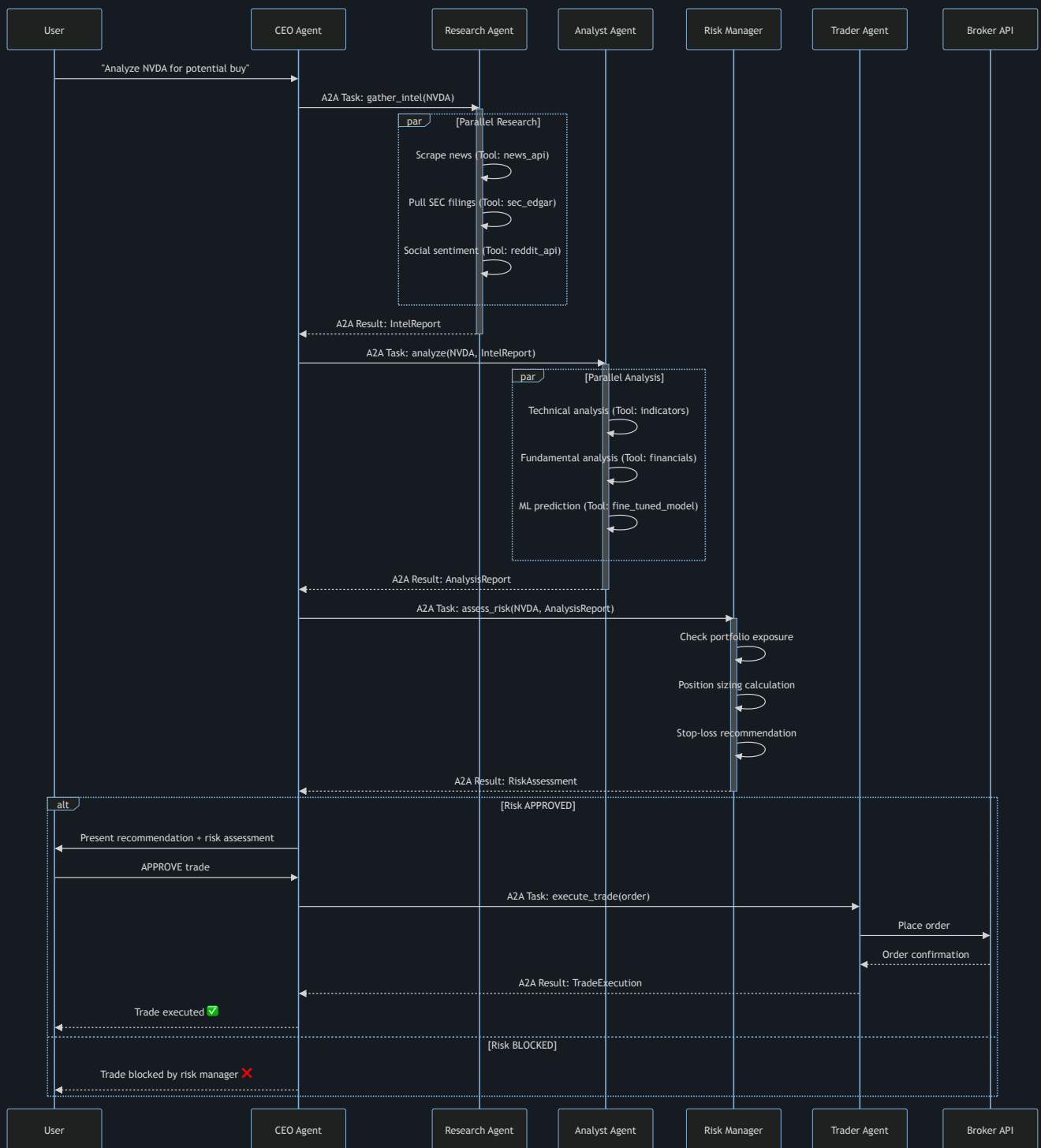
5.1 Design Decision Summary

Decision	Choice	Why
Frontend	Next.js 16 (deployed on Cloud Run)	SSR for SEO (public market pages); streaming for real-time data
Backend	Python 3.12 (FastAPI)	Best ecosystem for quantitative finance; NumPy/Pandas; ML libraries
Agent Framework	Google ADK (primary) + CrewAI (prototyping)	ADK is native GCP; CrewAI for rapid multi-agent iteration
Stateful Agents	LangGraph	Complex stateful workflows with human-in-the-loop approvals
Database	Firestore	Real-time listeners for portfolio updates; serverless
Analytics DB	BigQuery	Historical market data analysis; ML training data storage
Message Bus	Cloud Pub/Sub	Real-time market data fan-out; agent communication backbone
AI Model	Vertex AI (Gemini 2.5 Pro / Flash)	Native function calling; fine-tuning support; evaluation service
Fine-tuning	Vertex AI Fine-Tuning	Gemma models on historical data; reward-based RLHF
Knowledge Graph	Neo4j on GCP (Cloud Marketplace)	Company relationships; supply chains; sector correlations
Real-time Data	Cloud Run + Pub/Sub	WebSocket connections to broker APIs; fan-out to agents
Broker	Zerodha Kite Connect API	India's largest discount broker; REST + WebSocket APIs; ₹2000/month for market data; paper trading via Kite sandbox

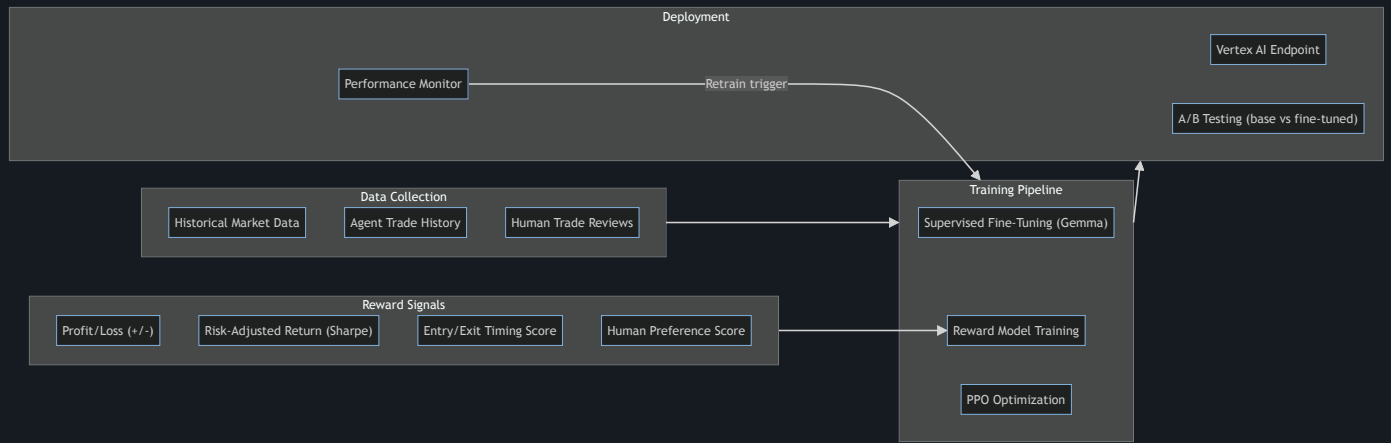
5.2 Multi-Agent Architecture



5.3 A2A Protocol Flow



5.4 RLHF Training Pipeline



5.5 Implementation Phases

Phase	Scope	Duration
P1: GCP Foundation	Terraform modules; Cloud Run; Firestore; Pub/Sub; IAM; basic dashboard UI	2-3 weeks
P2: Market Data Pipeline	Real-time data ingestion via Pub/Sub; BigQuery historical data load; data normalization	2 weeks
P3: Single Agent	Research Agent with ADK; function calling for news/SEC/sentiment APIs	2-3 weeks
P4: Multi-Agent Orchestration	CEO + Analyst + Risk agents; A2A protocol; task routing; state management	3-4 weeks
P5: Trading Execution	Broker API integration (paper trading first); order management; P&L tracking	2-3 weeks
P6: Knowledge Graph	Neo4j deployment; company/sector relationship mapping; graph-augmented analysis	2 weeks
P7: Fine-tuning & RLHF	Gemma fine-tuning on historical data; reward model; PPO optimization; A/B testing	4-6 weeks
P8: MCP Servers (x3)	Market Data, Portfolio, Trading MCP servers; test clients	2 weeks
P9: Dashboard & Polish	Real-time dashboard; alerting; E2E tests; paper trading validation	3 weeks

5.6 Acceptance Criteria

ID	Criterion	Verification
SS-AGENT-01	Orchestrator correctly delegates tasks to ≥ 3 specialized agents	E2E trace: verify A2A message flow for 10 analysis requests

ID	Criterion	Verification
SS-AGENT-02	Agents communicate via A2A protocol with proper task/result envelopes	Protocol conformance test against A2A spec
SS-AGENT-03	Risk Manager can block trade execution (hard guardrail)	Test: submit order exceeding position limit → blocked
SS-AI-01	Fine-tuned model outperforms base model on backtested returns by $\geq 5\%$	6-month backtest comparison; statistical significance test
SS-AI-02	RLHF-trained model shows improved Sharpe ratio vs supervised-only	A/B test over 1000 simulated trades
SS-AI-03	Function calling selects correct tools $\geq 98\%$ of the time	100-query automated evaluation
SS-RT-01	Market data updates reflected in dashboard within 2s	WebSocket latency measurement; 1000 data points
SS-RT-02	Agent decision pipeline completes within 30s for a single stock analysis	p95 latency across 50 analysis requests
SS-SEC-01	Paper trading mode cannot execute real orders	Integration test: verify Zerodha Kite sandbox mode; no live orders
SS-SEC-02	All agent decisions are logged with full reasoning chain	Audit log review: 100% of trade decisions have reasoning
SS-MCP-01	All 3 MCP servers respond correctly to their tool definitions	MCP test client: full tool coverage test

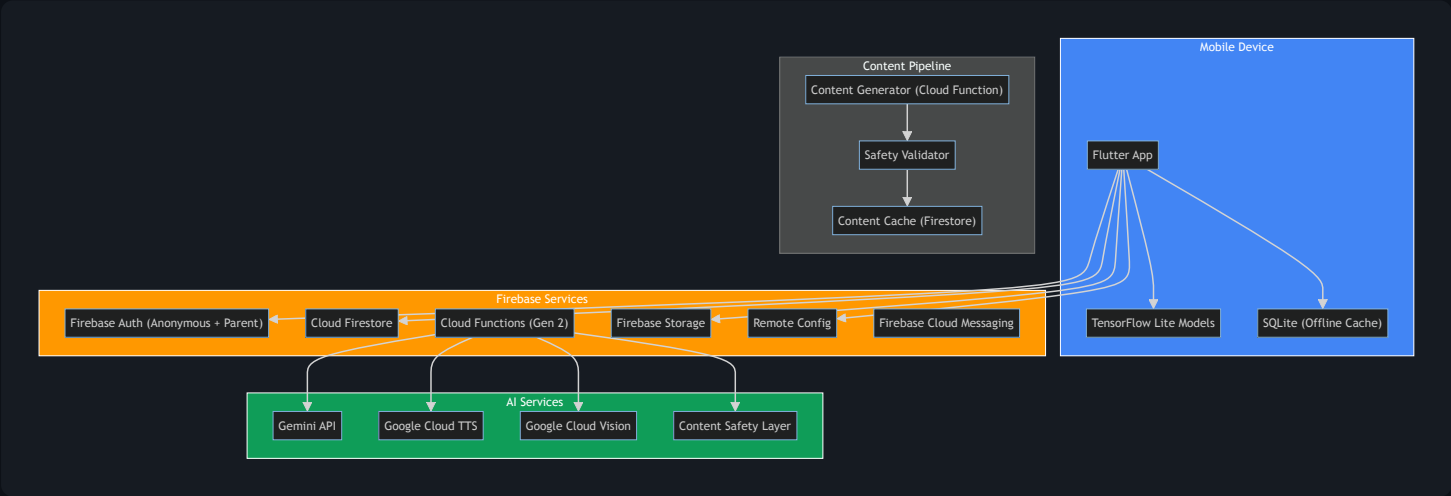
6. Project 4 — Kids App (GCP Firebase)

6.1 Design Decision Summary

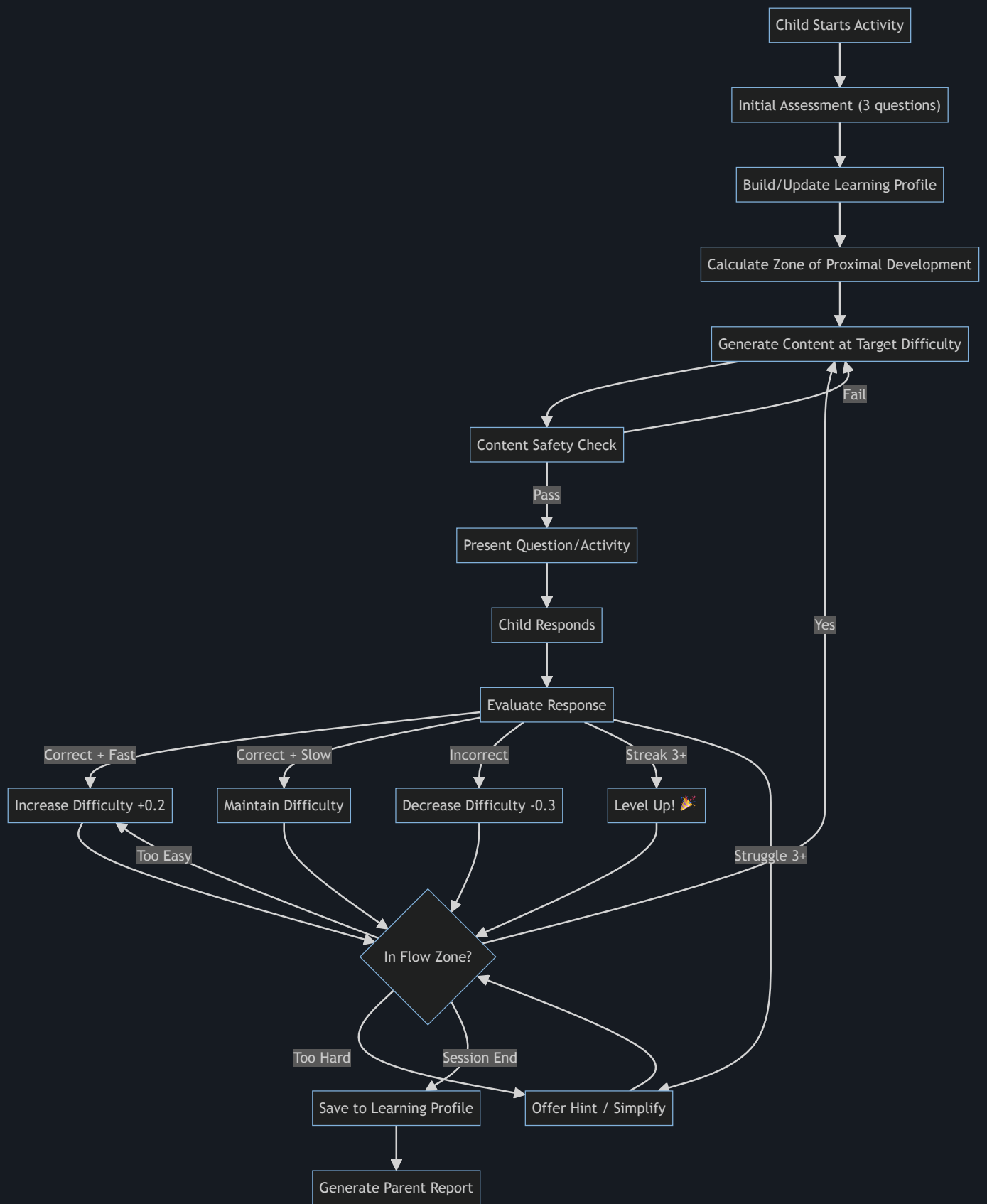
Decision	Choice	Why
Client	Flutter (iOS + Android + Web)	Fastest cross-platform framework; compiled to native ARM code; 120fps animations; Impeller rendering engine eliminates shader jank — critical for kids' attention span
Backend	Firebase (Firestore + Auth + Cloud Functions + Hosting)	Serverless; real-time sync; easy auth; Spark plan (free) covers development + early users

Decision	Choice	Why
AI Model	Gemini API (via Firebase Extensions or direct)	Content generation; adaptive difficulty; safety filters built-in
On-device ML	TensorFlow Lite + Firebase ML	Offline capabilities; drawing recognition; privacy for kids
Speech	Google Cloud Text-to-Speech	Read questions aloud; multiple voices; SSML support
Vision	Google Cloud Vision API	Object recognition for interactive learning
Content Safety	Gemini Safety Settings (BLOCK_LOW_AND_ABOVE) + Custom filters	COPPA compliance; age-appropriate content only
Analytics	Firebase Analytics (with consent)	Learning progress tracking; no PII collection

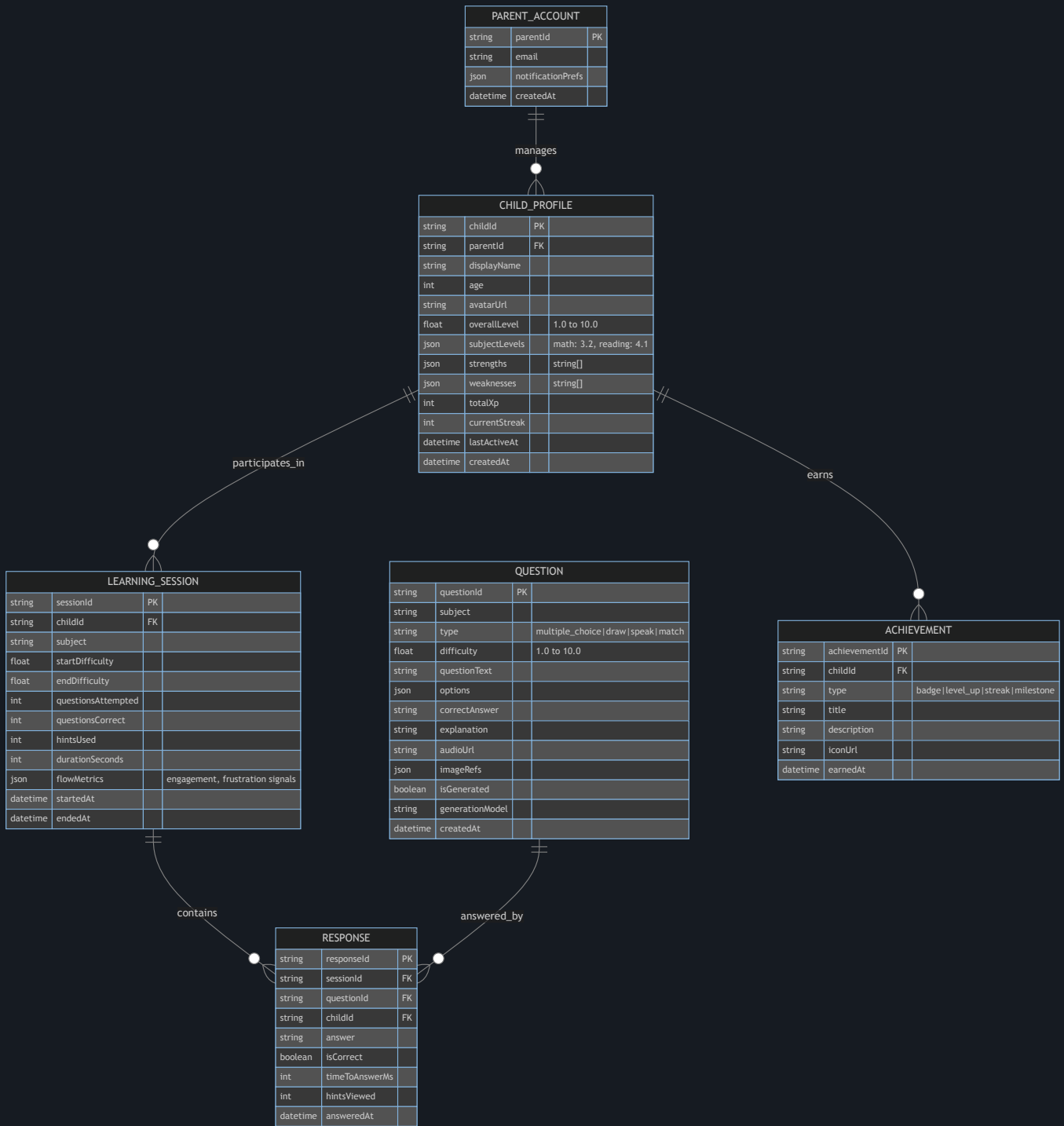
6.2 System Architecture



6.3 Adaptive Learning Algorithm



6.4 Data Model



6.5 Implementation Phases

Phase	Scope	Duration
P1: Flutter Foundation	App scaffold; Firebase setup; auth (anonymous + parent); Firestore schema; basic UI	2-3 weeks
P2: Core Learning Engine	Adaptive difficulty algorithm; question presentation; response evaluation; flow zone detection	3 weeks
P3: Content Generation	Gemini-powered question generation; safety filtering; content caching; multiple subjects	2-3 weeks

Phase	Scope	Duration
P4: Voice & Vision	TTS for questions; speech recognition for answers; camera-based drawing recognition	2 weeks
P5: Edge AI	TFLite models for offline drawing/object recognition; offline mode with cached content	3 weeks
P6: Gamification	XP system; achievements; streaks; avatars; parent dashboard with progress reports	2 weeks
P7: Safety & Compliance	COPPA compliance review; content safety hardening; parental controls; data minimization	2 weeks
P8: Polish	Animations; sound effects; performance optimization; accessibility for kids with disabilities	2 weeks

6.6 Acceptance Criteria

ID	Criterion	Verification
KA-LEARN-01	Adaptive algorithm adjusts difficulty within 3 questions of skill mismatch	Simulation: 100 synthetic learner profiles; verify convergence
KA-LEARN-02	Children stay in "flow zone" $\geq 70\%$ of session time	A/B test: adaptive vs fixed difficulty; measure engagement metrics
KA-LEARN-03	Generated questions are age-appropriate and factually correct	Human review: 200 generated questions rated by educators
KA-SAFETY-01	Zero inappropriate content reaches children	Adversarial content generation test (1000 prompts); Content Safety blocks 100%
KA-SAFETY-02	App collects zero PII from children without parental consent	Privacy audit; network traffic inspection; Firestore schema review
KA-EDGE-01	Core learning activities work fully offline	Airplane mode test: complete 10-question session without network
KA-EDGE-02	TFLite drawing recognition accuracy $\geq 85\%$ for target categories	Test with 500 child-like drawings across 20 categories
KA-VOICE-01	TTS reads questions clearly and at appropriate speed for age group	User test: 10 children ages 4-8; comprehension $\geq 90\%$
KA-PERF-01	App cold start $\leq 3s$ on mid-range Android device	Benchmark on Pixel 6a equivalent

ID	Criterion	Verification
KA-GAME-01	Gamification increases session duration by $\geq 20\%$	A/B test: gamified vs non-gamified; 100 sessions each

7. Project 5 — Folder Manager (Azure)

7.1 Design Decision Summary

Decision	Choice	Why
Frontend	Next.js 16 (React 19)	Consistent with Rahul Site; SSR for shared links; file preview rendering
Backend	Next.js API Routes + Azure Functions	API routes for CRUD; Functions for async processing (OCR, embedding)
Database	Cosmos DB for MongoDB (shared account, separate DB)	Reuse existing Azure Cosmos; cost-efficient
Storage	Azure Blob Storage	Already hosting files; tiered storage for cost optimization
AI - OCR	Azure AI Document Intelligence	PDF/image text extraction; table/form recognition; handwriting
AI - Search	Azure AI Search	Same as Rahul Site; vector + keyword hybrid
AI - Embeddings	Azure OpenAI text-embedding-3-large	Consistent embedding model across Azure projects
AI - Summarization	Azure OpenAI GPT-4o	Auto-summaries; auto-classification; natural language answers
Auth	NextAuth + Entra ID	Enterprise-ready; same-cloud SSO; multi-user support

7.2 System Architecture

7.4 Implementation Phases

Phase	Scope	Duration
P1: File Management Core	Upload/download/delete; folder structure; Blob storage; Cosmos metadata; auth	2 weeks
P2: Document Intelligence	Azure Function for OCR; PDF/image text extraction; table/form recognition	2 weeks
P3: Auto-Classification	GPT-4o powered classification; tag generation; confidence scoring	1-2 weeks
P4: Semantic Search	Azure AI Search index; embedding generation; hybrid search; natural language queries	2-3 weeks
P5: Smart Answers	RAG-style Q&A over file contents; citation of source files; streaming responses	2 weeks
P6: MCP Server	<code>files-mcp</code> exposing search + file retrieval tools	1 week
P7: Polish	File preview (PDF/image/code); sharing; storage tier management; E2E tests	2 weeks

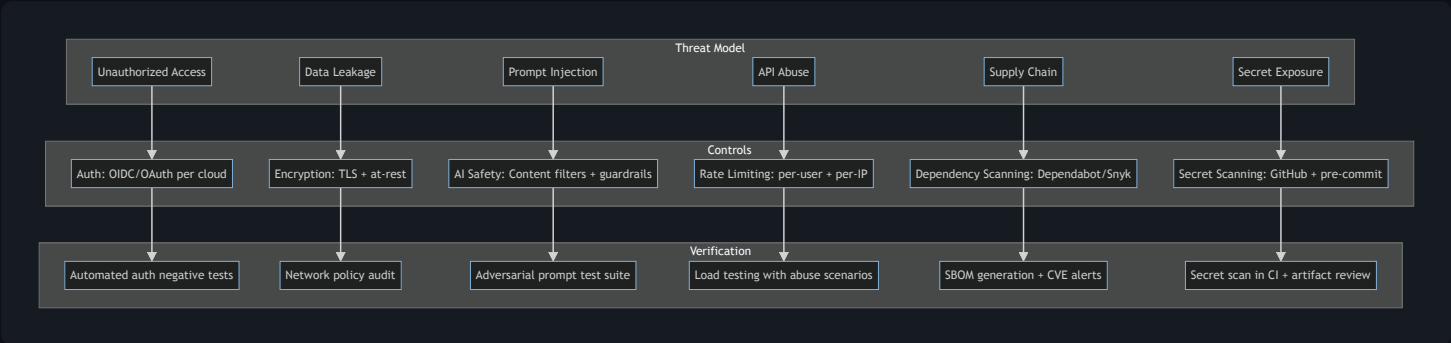
7.5 Acceptance Criteria

ID	Criterion	Verification
FM-CORE-01	Upload, browse, download, and delete files across folder hierarchy	E2E test: full file lifecycle
FM-CORE-02	Files up to 100 MiB upload successfully with progress indication	Boundary test: 99 MiB, 100 MiB, 101 MiB
FM-OCR-01	Text extraction from scanned PDFs with $\geq 95\%$ character accuracy	Test with 20 varied document types
FM-OCR-02	Processing completes within 60s for a 10-page PDF	p95 latency measurement
FM-CLASS-01	Auto-classification accuracy $\geq 85\%$ across 10 document categories	Test with 100 pre-labeled documents
FM-SEARCH-01	Semantic search: "laptop purchase" returns "MacBook Pro order" documents	30-query semantic similarity test; precision@5 ≥ 0.7
FM-SEARCH-02	Natural language answers cite the correct source file $\geq 90\%$ of the time	Evaluation set: 50 questions with known answers
FM-SEC-01	Users can only access their own files; no cross-tenant data leakage	Multi-user isolation test

ID	Criterion	Verification
FM-MCP-01	MCP <code>search_files</code> tool returns relevant results for 10 test queries	MCP test client verification

8. Cross-Cutting Concerns

9.1 Security Architecture (All Projects)



9.2 Observability Standard

Every project implements this observability stack:

Layer	Azure Projects	AWS Projects	GCP Projects
Logging	Application Insights + structured JSON	CloudWatch Logs	Cloud Logging
Metrics	App Insights Metrics	CloudWatch Metrics	Cloud Monitoring
Tracing	App Insights Distributed Tracing	X-Ray	Cloud Trace
Alerting	Azure Monitor Alerts	CloudWatch Alarms	Cloud Alerting
Dashboard	Azure Workbook	CloudWatch Dashboard	Looker Studio

Unified Log Schema (JSON)

```
{
  "timestamp": "ISO-8601",
  "service": "rahulsite|creatorvault|stocksite|kidsapp|foldermanager",
  "requestId": "uuid-v4",
  "operation": "search|upload|chat|trade|...",
  "userId": "hashed-id (no PII)",
  "duration_ms": 142,
  "status": "success|error|timeout",
}
```

9.3 Testing Strategy



```

infrastructure-deployments/
├── modules/
│   └── # Reusable, tested modules
│       ├── azure/
│       │   ├── webapp/
│       │   │   └── # App Service + deployment slots
│       │   ├── cosmos-db/
│       │   │   └── # MongoDB + Gremlin APIs
│       │   ├── ai-search/
│       │   │   └── # Azure AI Search instance
│       │   ├── openai/
│       │   │   └── # Azure OpenAI deployment
│       │   ├── function-app/
│       │   │   └── # Consumption/Premium plan functions
│       │   ├── storage/
│       │   │   └── # Blob + lifecycle policies
│       │   ├── keyvault/
│       │   │   └── # Secrets + access policies
│       │   ├── container-app/
│       │   │   └── # Container Apps Environment
│       │   ├── content-safety/
│       │   │   └── # Azure AI Content Safety
│       │   ├── document-intelligence/
│       │   │   └── # Azure AI Document Intelligence
│       │   └── network/
│       │       └── # VNet + subnets + NSG + PE
│       ├── aws/
│       │   ├── lambda/
│       │   │   └── # Function + layers + aliases
│       │   ├── ecs-fargate/
│       │   │   └── # Task definition + service
│       │   ├── s3/
│       │   │   └── # Buckets + lifecycle + replication
│       │   ├── dynamodb/
│       │   │   └── # Tables + GSI + DAX
│       │   ├── api-gateway/
│       │   │   └── # REST + WebSocket APIs
│       │   ├── step-functions/
│       │   │   └── # State machines
│       │   └── bedrock/
│       │       └── # Model access + prompt caching

```

```

├── cognito/                # User pools + identity pools
├── cloudfront/             # Distribution + OAI
├── vpc/                    # VPC + subnets + NAT + SG
├── gcp/
│   ├── cloud-run/         # Service + revisions + traffic
│   ├── firestore/        # Database + indexes + rules
│   ├── bigquery/          # Dataset + tables + views
│   ├── cloud-functions/   # Gen 2 functions
│   ├── pub-sub/           # Topics + subscriptions + DLQ
│   ├── vertex-ai/         # Endpoints + fine-tuning jobs
│   ├── firebase/          # Project + apps + extensions
│   ├── neo4j/             # Marketplace deployment
│   └── vpc/               # VPC + subnets + firewall
├── rahulsite/              # Project 1 – Azure
│   ├── main.tf            # Module composition
│   ├── variables.tf        # Input variables
│   ├── outputs.tf          # Exported values
│   ├── backend.tf          # Azure Storage state
│   └── terraform.tfvars    # Environment values
├── foldermanager/          # Project 5 – Azure
│   └── ... (same structure)
├── creatorvault/           # Project 2 – AWS
│   ├── main.tf
│   ├── backend.tf         # S3 state backend
│   └── ...
├── stocksite/              # Project 3 – GCP
│   ├── main.tf
│   ├── backend.tf         # GCS state backend
│   └── ...
├── kidsapp/                # Project 4 – GCP Firebase
│   └── ... (same structure)
├── .github/workflows/
│   ├── infra-azure.yml     # rahulsite + foldermanager
│   ├── infra-aws.yml       # creatorvault
│   └── infra-gcp.yml       # stocksite + kidsapp
└── README.md

```

9.2 State Management

Cloud	Backend	Bucket/Container	Lock
Azure	azurerm	Storage Account: tfstaterahultech	Blob lease
AWS	s3	S3: tfstate-creatorvault-{account-id}	DynamoDB
GCP	gcs	GCS: tfstate-stocksite-{project-id}	Native

9.3 Cost Guardrails (Cloud-Optimized)

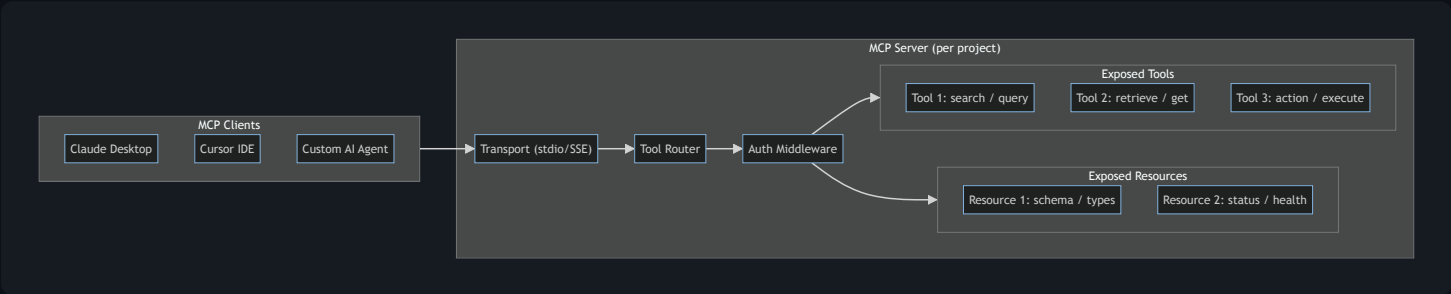
Project	Cloud	Monthly Target	Free Tier Leverage	Key Cost Drivers	Optimization Strategy
Rahul Site	Azure	≤ ₹3,500 (~\$42)	Cosmos DB free tier (1000 RU/s, 25 GB); App Service F1/B1	OpenAI tokens, AI Search	Cache AI responses aggressively; batch embed on publish only; use AI Search free tier (50 MB / 3 indexes)
Folder Manager	Azure	≤ ₹2,500 (~\$30)	Shares Cosmos account with Rahul Site; shared AI Search	Document Intelligence, OpenAI	Batch OCR in off-peak; cache embeddings; use consumption Function App (1M free executions/month)
Creator Vault	AWS	≤ ₹2,000 (~\$24)	Lambda free tier (1M requests/month); S3 first 5 GB free; DynamoDB 25 GB free	Fargate tasks, Bedrock tokens, Transcribe	S3 Intelligent-Tiering; Fargate Spot for downloads; prompt caching (40-60% LLM cost reduction); Transcribe on-demand pricing
Stock Site	GCP	≤ ₹6,500 (~\$78)	Cloud Run free tier (2M requests/month); Firestore free (1 GiB); BigQuery free (1 TB query/month)	Vertex AI fine-tuning, Pub/Sub, Zerodha API (₹2000/month)	Cloud Run min-instances=0; BigQuery partitioned tables; use Gemini Flash (not Pro) for routine tasks; model distillation for inference
Kids App	GCP Firebase	≤ ₹1,200 (~\$15)	Firebase Spark plan: Firestore 1 GiB, Auth 10K/month, Functions 2M	Gemini API, Cloud TTS	Pre-generate question batches (not per-request); TFLite for all offline ML; TTS

Project	Cloud	Monthly Target	Free Tier Leverage	Key Cost Drivers	Optimization Strategy
			invocations, Hosting 10 GB		audio caching in Firebase Storage; use Gemini Flash for generation

10. MCP Integration Architecture

11.1 MCP Server Design Pattern

Every MCP server follows this structure:



11.2 MCP Server Registry

Server	Transport	Tools	Resources
rahul-knowledge-mcp	stdio	search_documents , ask_question , list_topics	schema://documents , status://health
creatorvault-mcp	stdio	search_transcripts , get_summary , list_videos	schema://videos , status://health
market-data-mcp	SSE	get_stock_quote , get_historical , search_news	schema://market , status://health
portfolio-mcp	SSE	get_holdings , get_pnl , get_trade_history	schema://portfolio , status://health
trading-mcp	SSE	place_order , cancel_order , get_order_status	schema://orders , status://health
files-mcp	stdio	search_files , get_file_info , download_file	schema://files , status://health

11. Master Build Sequence



Build Order Rationale

Priority	Project	Reason
1	Rahul Site	90% existing code; teaches RAG + embeddings + search — foundational AI concepts needed everywhere
2	Folder Manager	Shares Azure infra + AI services with Rahul Site; reuses search/embedding patterns
3	Creator Vault	Introduces AWS; serverless patterns; multimodal AI — new territory with manageable scope
4	Stock Site	Most complex; multi-agent + fine-tuning + Zerodha integration — needs mature AI skills from previous projects
5	Kids App	Fun capstone; edge AI + Flutter — leverages GCP knowledge from Stock Site

Owner Decisions Log

Date	Decision	Resolution
2026-10-01	PG Site	Dropped — 5 projects total
2026-10-01	Stock broker	Zerodha Kite Connect — India market; ₹2000/month data subscription
2026-10-01	Kids App client	Flutter — Fastest cross-platform; native ARM compilation; Impeller engine
2026-10-01	Cost strategy	Cloud-native free tiers first — INR-denominated budgets; total ₹15,700/month (\$189) across all 5 projects

[!TIP] **Recommended next step:** Start with Rahul Site Phase P1 (Foundation Fix) — closing the 10 gaps identified in the existing [LLD](#). This is the highest-impact, lowest-risk starting point.

This document is a living architecture baseline. Update it with each major design decision. Version changes must include rationale, affected acceptance criteria, and updated diagrams.